

AI Recruitment Platform

CV Review · Technical Interview · Soft Skills Interview · Final AI Review

Project Documentation

Backend	NestJS
Database	Neon PostgreSQL + Prisma ORM
Frontend	React + Tailwind (already built)
AI	OpenAI — GPT, Embeddings, Realtime API
Timeline	12 weeks (3 months)
Version	1.0

1. Project Overview

An AI recruitment platform that takes a candidate through four connected stages and gives the recruiter one evidence-backed report at the end.

Stage	What it does	Output
1. CV Review	Matches the CV against the job description using GPT extraction plus embeddings.	CV match score + missing skills
2. Technical Interview	Adaptive AI voice interview with a whiteboard and a code editor.	Technical + problem-solving scores
3. Soft Skills Interview	Same interview engine, behavioural agent and criteria.	Communication + behavioural scores
4. Final AI Review	Combines all three stages into one report.	Overall score + recommendation

The four stages share one candidate record, one job record and one scoring and reporting layer — a single pipeline, not three separate tools. The AI scores and explains; the recruiter decides.

2. Introduction

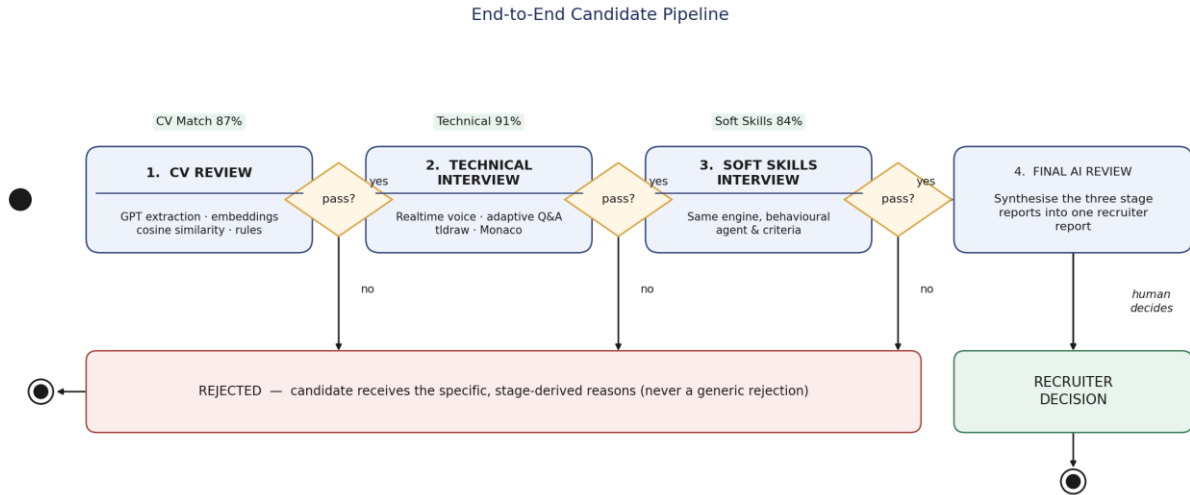
A single job post can attract hundreds of applications. Recruiters skim CVs in seconds, interview only a handful of people, and record almost nothing about why anyone was rejected. This platform automates the screening and interviewing work while keeping the reasoning visible and the final decision human.

3. Problem Statement

- Too many applications for a human team to read properly.
- Good candidates are missed because their CV wording differs from the job post.
- Interviewing every qualified candidate is too slow and too expensive.
- Rejections are unexplained and leave no auditable record.

4. Proposed Solution

A four-stage pipeline. Each stage produces a score and written reasoning, and gates entry to the next stage. A candidate rejected at any stage receives the specific reasons from that stage.



The four stages share one candidate record, one job record and one scoring and reporting layer — a single connected pipeline, not three separate AI tools.

Figure 1 — End-to-end candidate pipeline

5. Objectives

- Score every CV against the job description automatically, with explained reasoning.
- Run an adaptive AI technical interview with live voice, a whiteboard and a code editor.
- Reuse the same interview engine for the soft-skills stage.
- Combine all stages into one recruiter report.
- Keep the AI as decision support — every hire, waitlist or reject is a human action.
- Record integrity events during interviews as advisory signals only.
- Build the whole MVP on free tiers plus the existing OpenAI credit.

6. Users and Actors

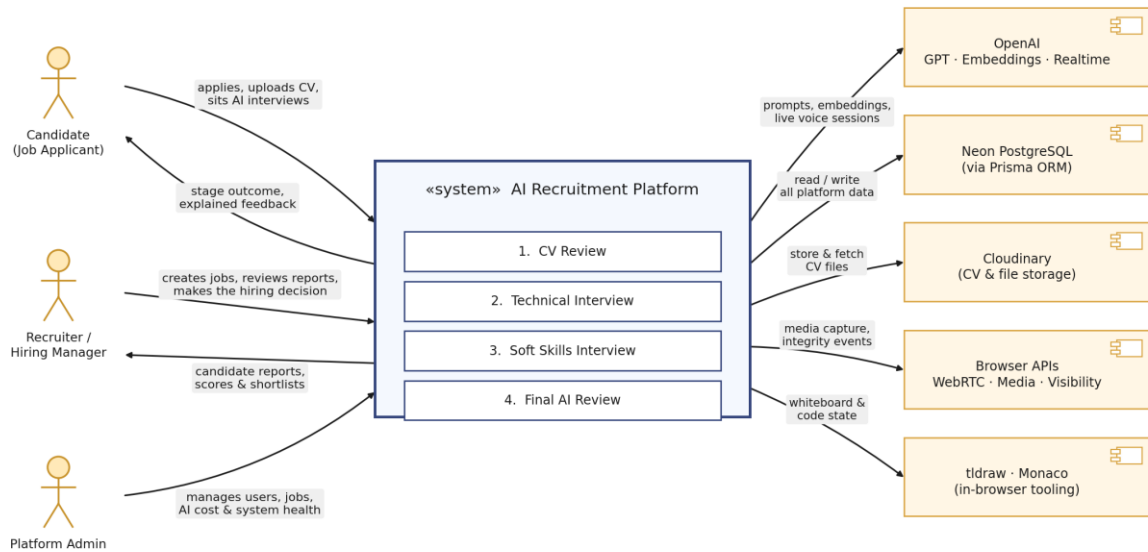


Figure 2 — Users, actors and external systems

Actor	Role
Candidate	Applies, uploads a CV, sits the AI interviews, sees the outcome.
Recruiter / Hiring Manager	Creates jobs, reviews reports, makes the hiring decision.
Platform Admin	Manages users, jobs, AI cost and system health.
OpenAI	GPT, Embeddings and Realtime API.
Neon PostgreSQL	All platform data, accessed through Prisma.
Cloudinary	CV and file storage.
Browser APIs	Media capture and integrity signals.

7. System Architecture

Four layers: the existing React client, a NestJS API, OpenAI as the AI service layer, and Neon PostgreSQL with Cloudinary for storage.

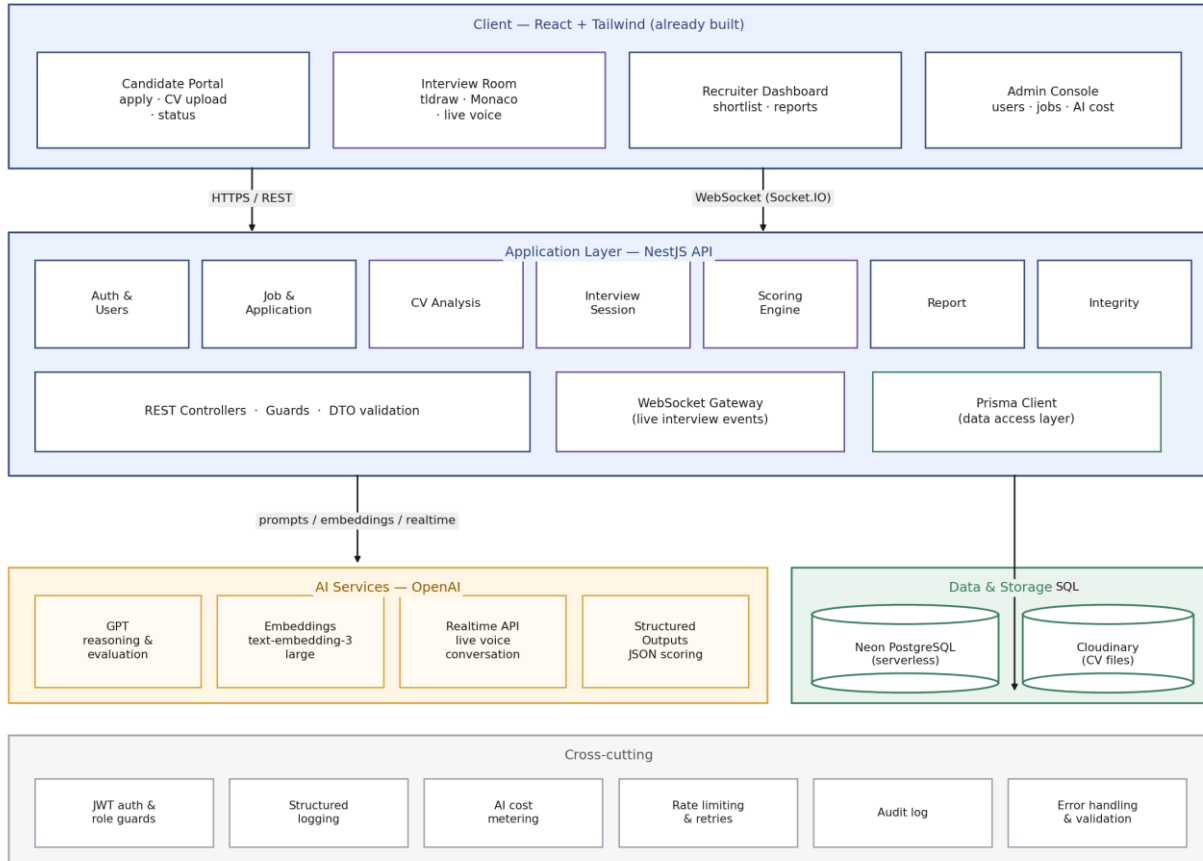


Figure 3 — System architecture

7.1 Backend Architecture (NestJS)

Two request paths: normal REST for CRUD and CV work, and a Socket.IO gateway for the live interview.

NestJS Backend Architecture

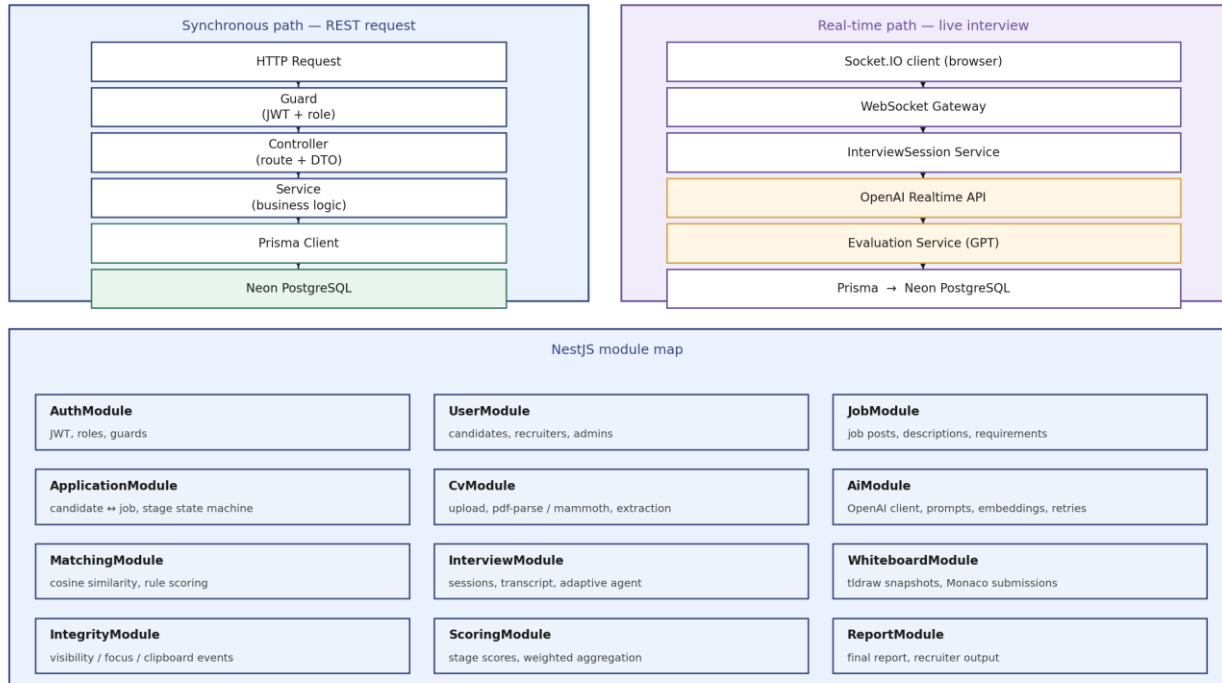


Figure 4 — NestJS backend architecture and module map

7.2 Frontend

Screen	Purpose	Key libraries
Candidate Portal	Apply, upload CV, track stage	React, Tailwind
Interview Room	AI interviewer + problem-solving area	tldraw, Monaco Editor, Socket.IO client
Recruiter Dashboard	Shortlist, scores, final reports	React, Tailwind
Admin Console	Users, jobs, AI cost, health	React, Tailwind

8. AI Integration

All AI calls go through one adapter so prompts, retries and cost metering stay in one place. The weighting of scores and the pass/fail thresholds are ordinary code, not model output.

AI Integration Map — which model does what

Pipeline stage	OpenAI capability used	Supporting tools	Produced artefact
1. CV Review	GPT — structured extraction of CV and Job Description Embeddings (text-embedding-3-large) — semantic vectors	pdf-parse (PDF) mammoth (DOCX) Cosine similarity Cloudinary storage	CvAnalysis: skills %, experience %, education %, projects %, certs, missing skills, summary, recommendation
2. Technical Interview	Realtime API — live voice conversation GPT — adaptive follow-ups, reasoning, evaluation	tidraw (whiteboard) Monaco (code) Socket.IO events Browser integrity APIs	InterviewSession + transcript, whiteboard snapshots, code submissions, technical score
3. Soft Skills Interview	Realtime API — live voice conversation GPT — behavioural reasoning and evaluation	Same interview engine (agent + criteria swapped) Socket.IO events	InterviewSession + transcript, communication / teamwork / leadership / adaptability scores
4. Final AI Review	GPT with Structured Outputs — synthesis across the three stage reports	Weighted scoring rules (deterministic, in code)	FinalReport: overall score, strengths, gaps, highlights, recommendation for a human

AI produces scores and explanations; the deterministic weighting and the final hiring decision stay outside the model.

Figure 5 — AI integration map

8.1 Stage 1 — CV Review

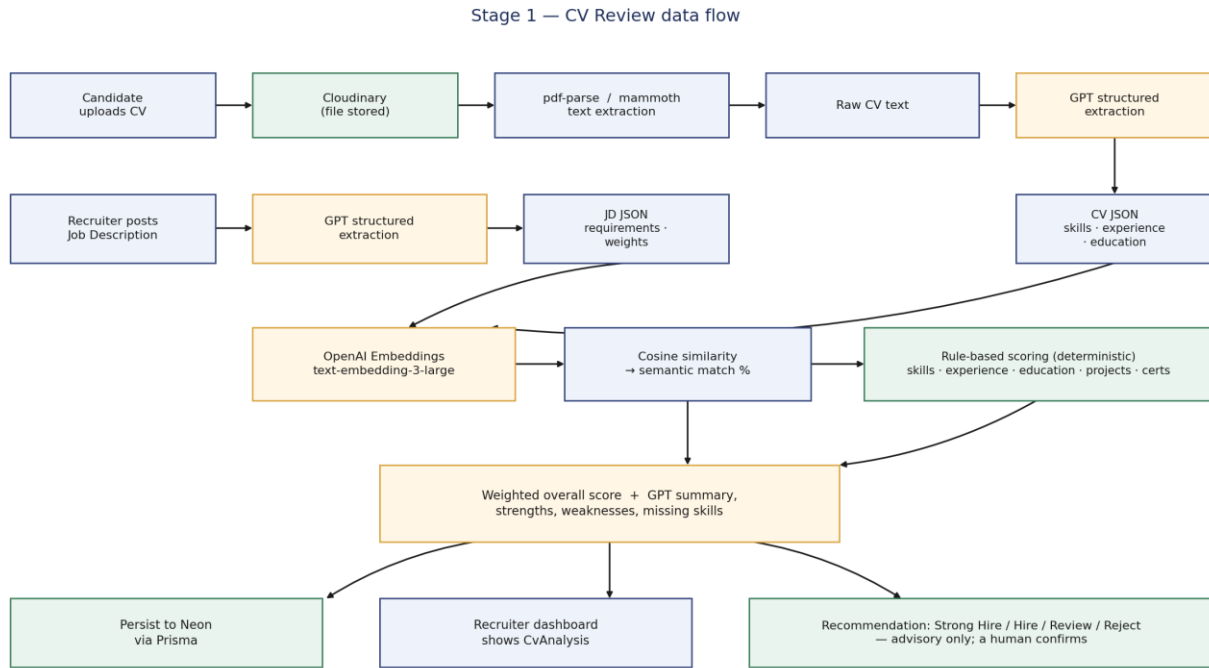


Figure 6 — CV Review data flow

Output field	Meaning
overallScore	Weighted match against the job description
skillsScore / experience / education	Per-dimension match
missingSkills	Required skills not evidenced in the CV
summary	Short AI-written explanation
recommendation	Strong Hire / Hire / Review / Reject — advisory

8.2 Stage 2 — Technical Interview

Two screens: the AI interviewer on the left, a tldraw whiteboard or Monaco editor on the right. The agent asks a question, listens, then chooses the next question from the answer — it is not a fixed question list.

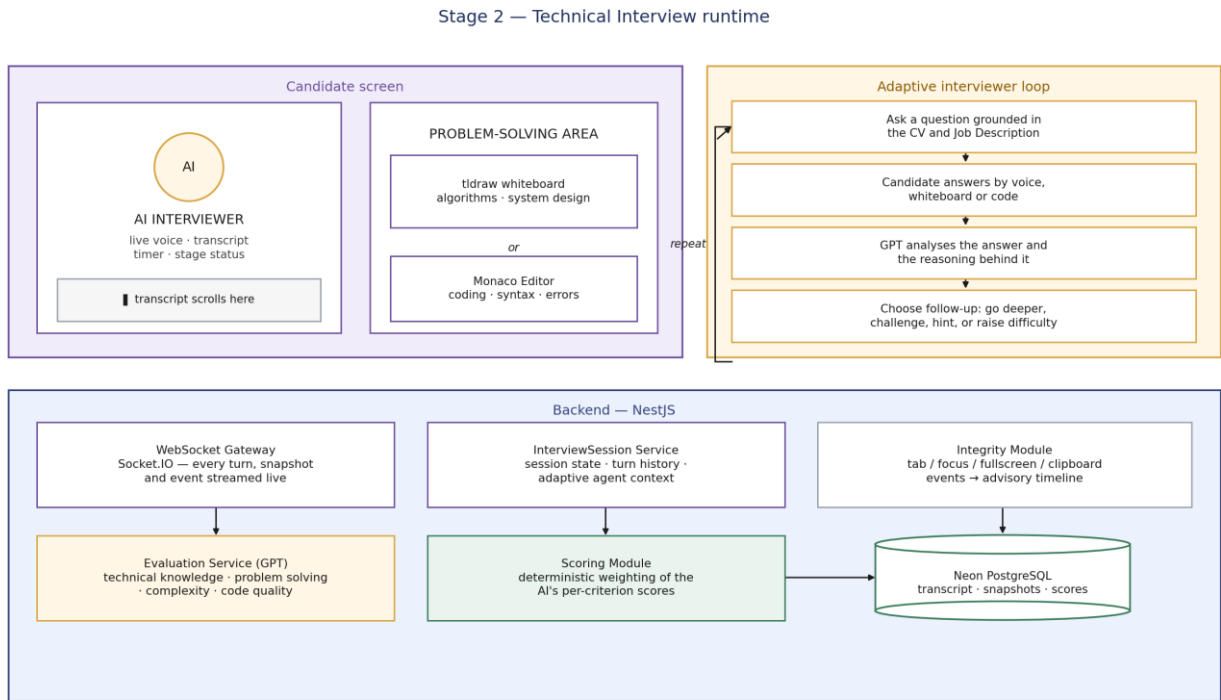


Figure 7 — Technical interview runtime

Evaluated	Integrity signals recorded (advisory only)
Technical knowledge Problem solving Code correctness Time & space complexity System design Code quality Communication while reasoning	Tab / window change Fullscreen exit Focus loss Clipboard activity These are recorded on a timeline for a human to interpret. They never reject a candidate on their own, and browser security limits what can be detected.

8.3 Stage 3 — Soft Skills Interview

The same session engine, transcript store, scoring and reporting are reused. Only the agent persona and the evaluation criteria change: communication, teamwork, leadership, adaptability, conflict resolution and clarity.

8.4 Stage 4 — Final AI Review

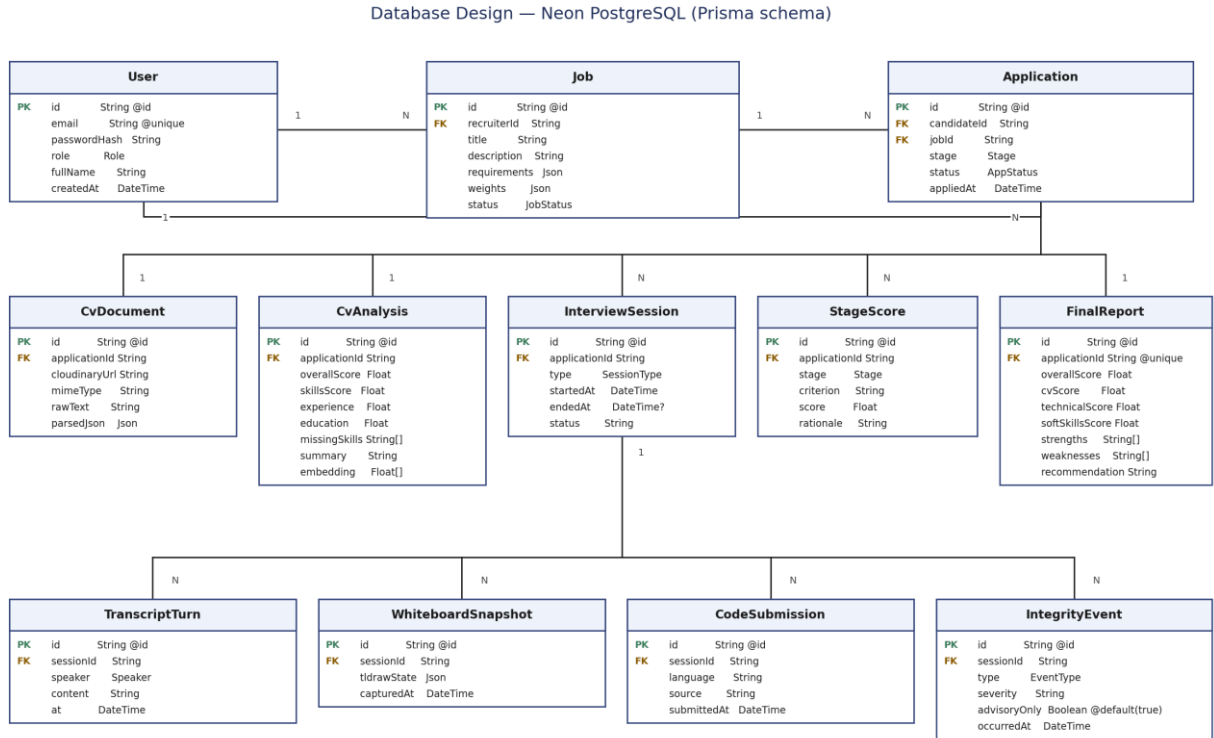
The three stage reports are combined into one recruiter-facing report.

Section	Contents
Scores	Overall, CV match, technical, problem solving, communication, soft skills
Evidence	Strengths, weaknesses, missing skills, interview highlights

Summary	AI-written narrative across all three stages
Recommendation	For recruiter review — never an automatic decision

9. Database Design

Neon PostgreSQL accessed through Prisma. Application is the hub — one row per candidate per job, carrying the current pipeline stage.



PK = primary key · FK = foreign key. Application is the hub: one row per candidate per job, carrying the pipeline stage. CV files live in Cloudinary; only the URL is stored.

Figure 8 — Database design (Prisma schema)

Table	Purpose
User	Candidates, recruiters and admins with a role
Job	Job post, requirements and scoring weights
Application	Candidate ↔ job, current stage and status
CvDocument / CvAnalysis	Uploaded CV and its analysis, scores and embedding
InterviewSession	One technical or soft-skills session
TranscriptTurn	Each turn of the conversation
WhiteboardSnapshot / CodeSubmission	tldraw state and code submitted
IntegrityEvent	Advisory integrity signals
StageScore	Per-criterion score for a stage
FinalReport	Combined report for the recruiter

10. System Models

10.1 Class Diagram

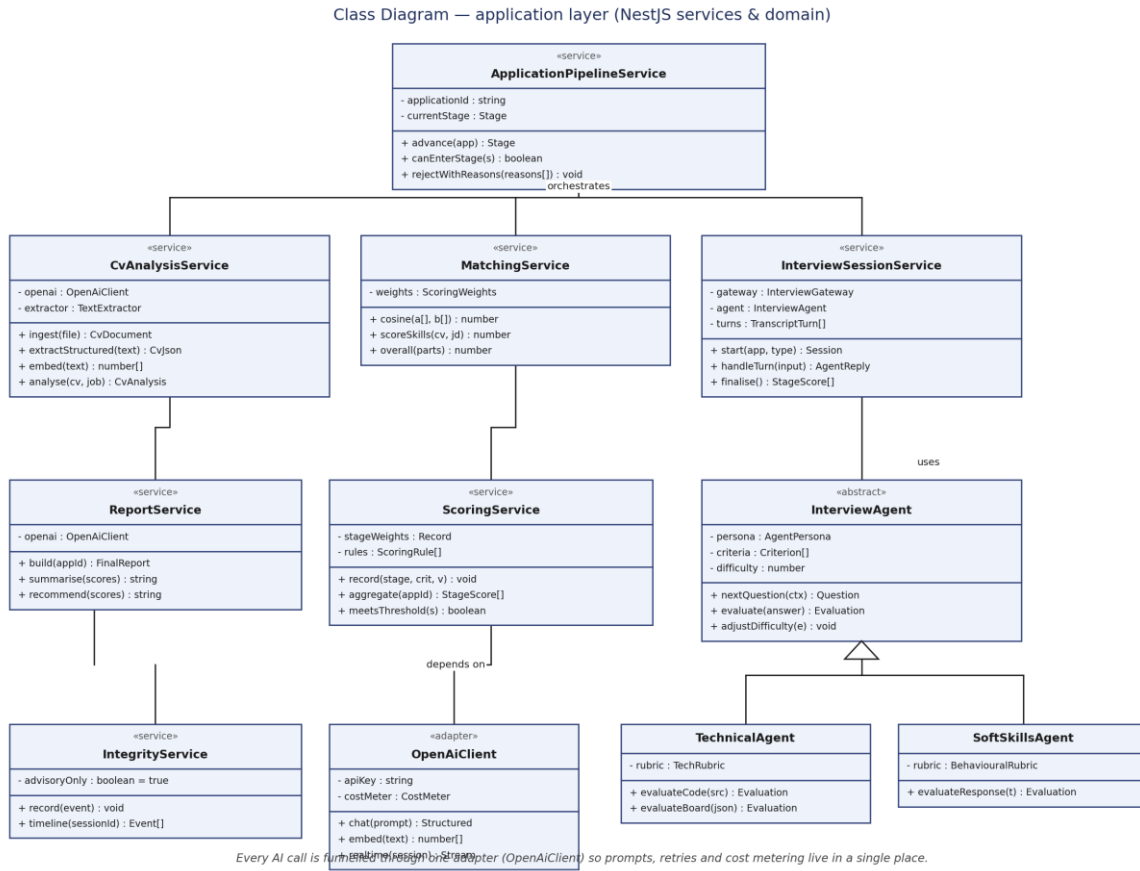


Figure 9 — Class diagram (services and domain)

10.2 Sequence — CV Review

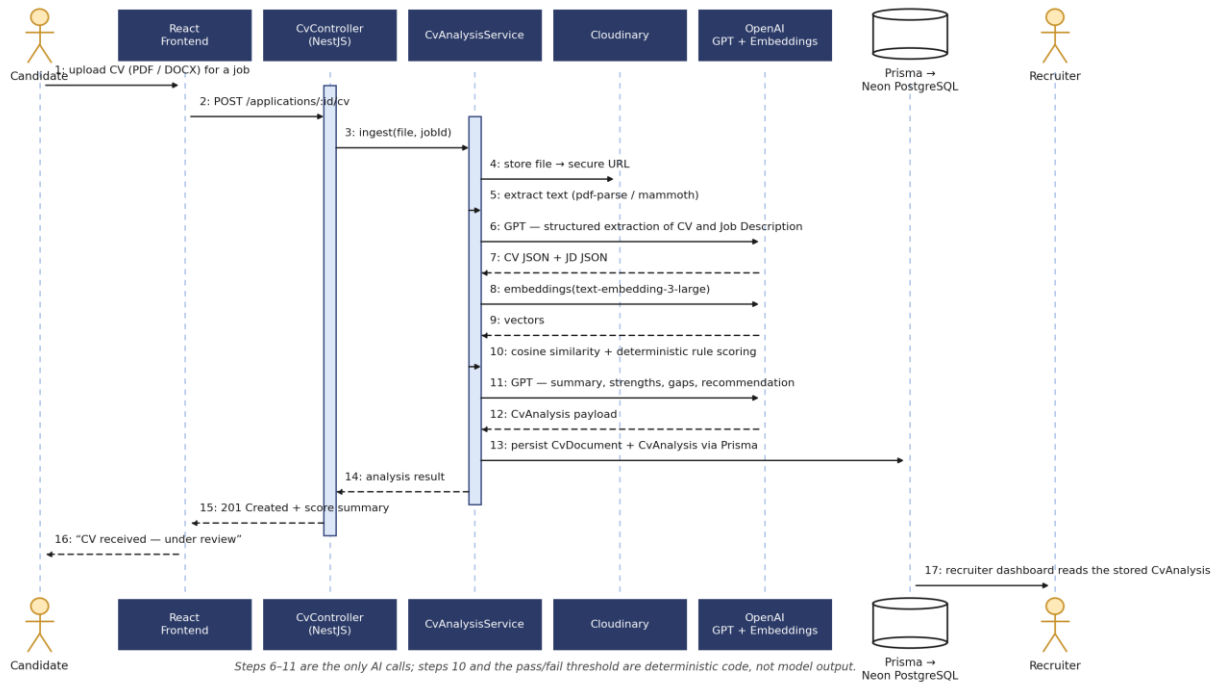


Figure 10 — Sequence: CV upload to analysis

10.3 Sequence — Technical Interview

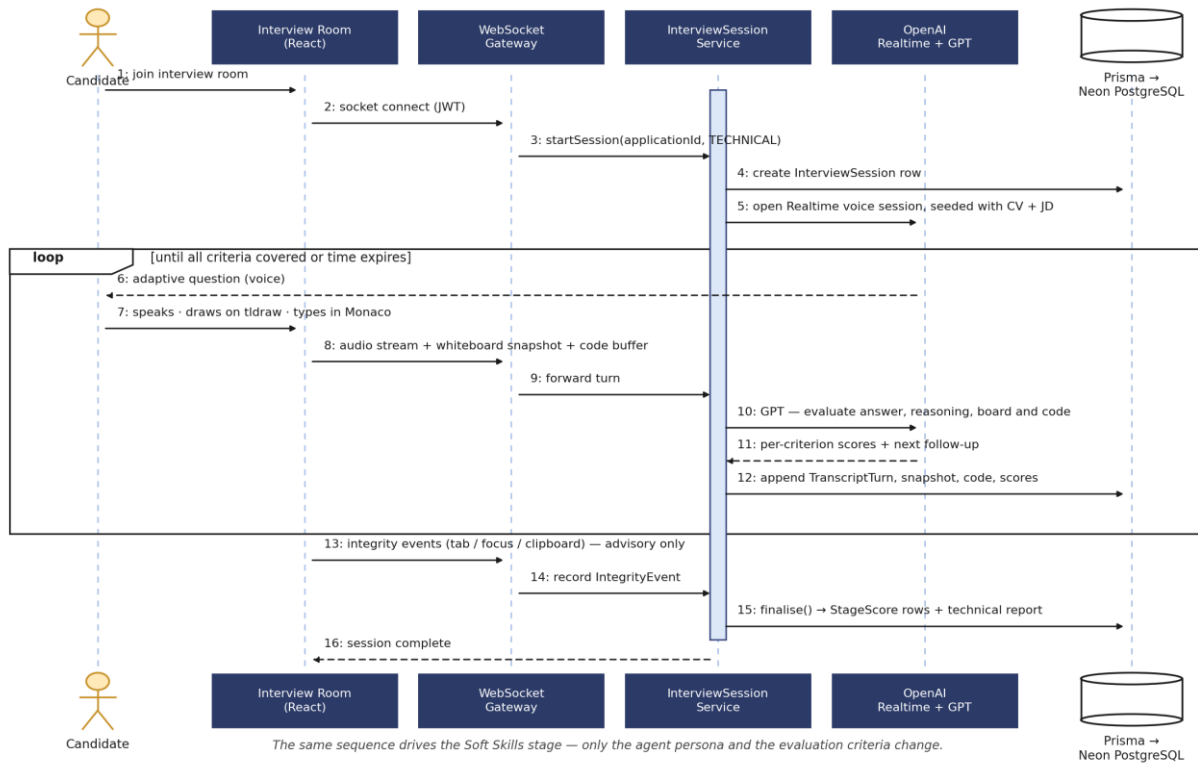


Figure 11 — Sequence: live technical interview

10.4 Workflow

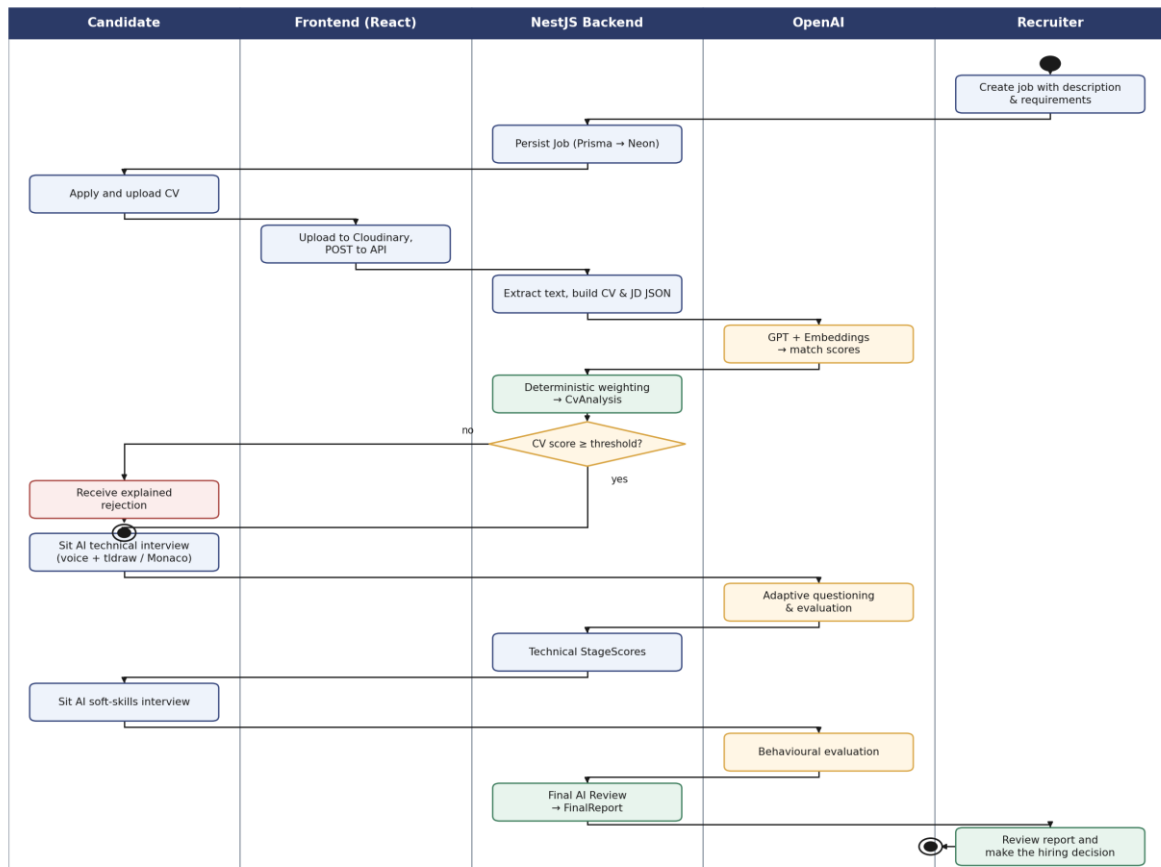


Figure 12 — End-to-end workflow

10.5 Application States

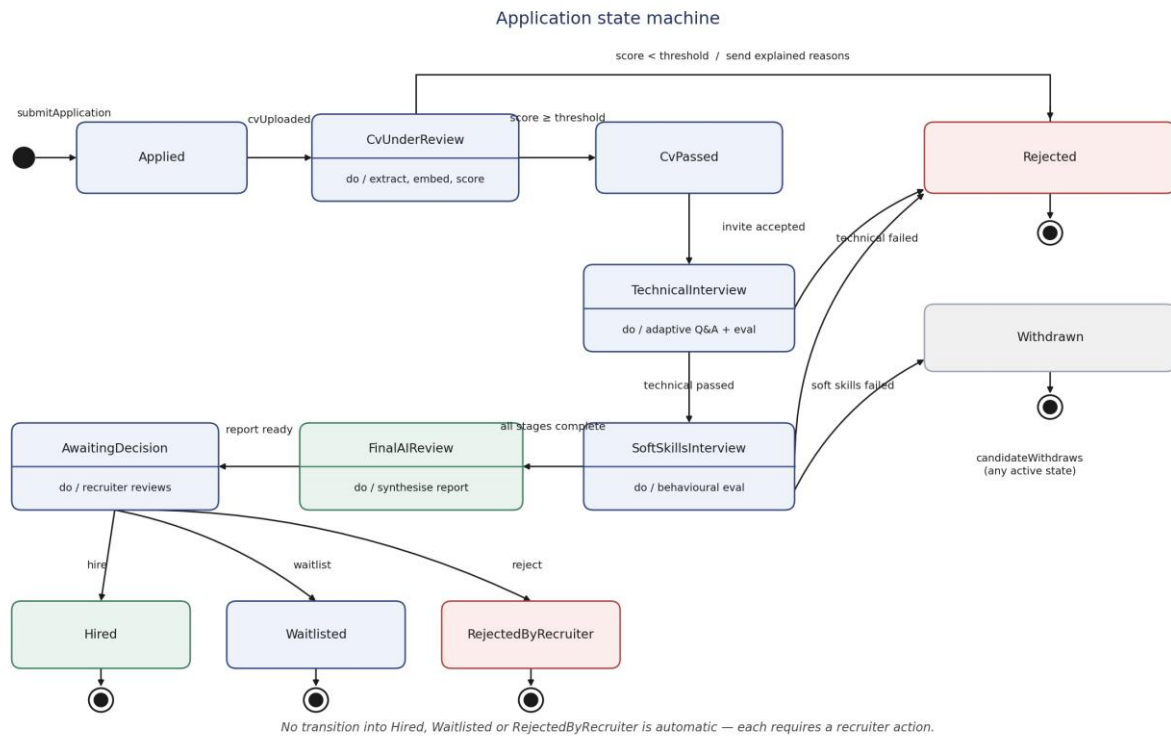


Figure 13 — Application state machine

11. Implementation

Build in stage order. The frontend and dashboard already exist and are not rebuilt.

Step	Build	Done when
1	CV Review — upload, extraction, GPT/JD analysis, embeddings, matching, scoring	A recruiter sees a scored CV with reasons
2	Technical Interview — session, Realtime voice, adaptive agent, tldraw, Monaco, evaluation	A candidate completes a scored technical interview
3	Soft Skills — reuse the engine with a behavioural agent	A soft-skills report is produced from the same pipeline
4	Final Review — aggregate the three stages into one report	The recruiter dashboard shows one combined report
5	Premium extras — realistic avatar, managed video, richer integrity checks	Only after the core pipeline is validated with real users

11.1 Backend Modules

Module	Responsibility
AuthModule	JWT, roles, guards
UserModule / JobModule	Users, job posts and requirements
ApplicationModule	Candidate ↔ job record and the stage state machine
CvModule	Upload, pdf-parse / mammoth, structured extraction
AiModule	OpenAI client, prompts, embeddings, retries, cost metering
MatchingModule	Cosine similarity and rule-based scoring
InterviewModule	Sessions, transcript, adaptive agent
WhiteboardModule	tldraw snapshots and Monaco submissions
IntegrityModule	Visibility, focus and clipboard events
ScoringModule / ReportModule	Stage scores and the final report

12. Technology Stack

Layer	Technology	Cost
Frontend	React + Tailwind (existing)	Already built
Whiteboard	tldraw	Free
Code editor	Monaco Editor	Free
Real-time	Socket.IO / WebSocket	Free
Media & integrity	WebRTC and browser APIs, MediaPipe (optional)	Free
Backend	NestJS	Free
ORM	Prisma	Free
Database	Neon PostgreSQL	Free tier
File storage	Cloudinary	Free tier
AI reasoning	OpenAI GPT	Existing \$100 credit
Live voice	OpenAI Realtime API	Existing credit
Matching	OpenAI Embeddings (text-embedding-3-large) + cosine similarity	Existing credit
CV parsing	pdf-parse, mammoth	Free

Additional MVP cost: approximately \$0. No avatar service or managed video platform is purchased until the core pipeline is proven. HeyGen (realistic avatar) and LiveKit (managed real-time infrastructure) are later upgrades.

13. Development Schedule

Twelve weeks, three months.

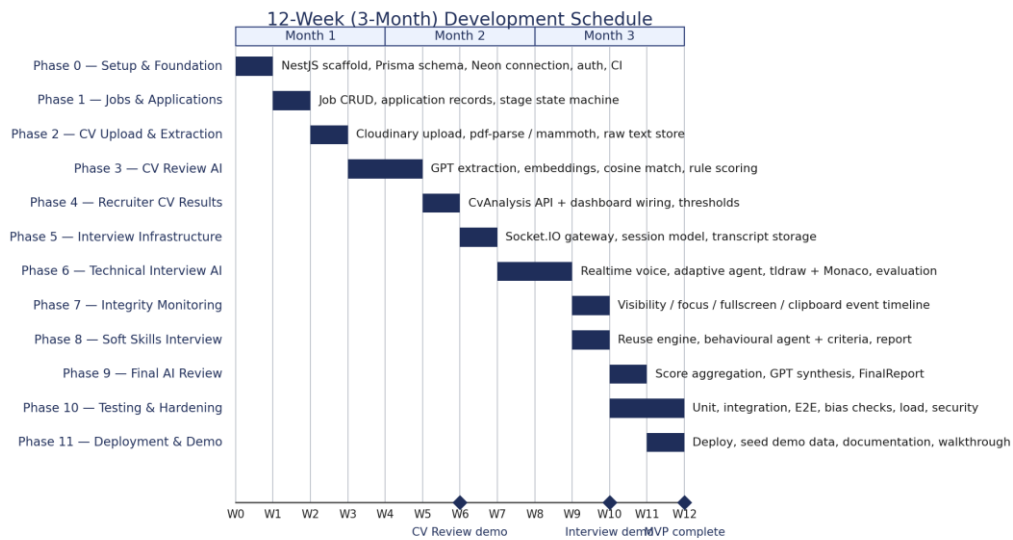


Figure 14 — 12-week development schedule

Weeks	Phase	Deliverable
0–1	Setup & foundation	NestJS + Prisma + Neon running, auth, CI
1–2	Jobs & applications	Job CRUD and the application stage machine
2–3	CV upload & extraction	Cloudinary upload and text extraction
3–5	CV Review AI	GPT extraction, embeddings, matching, scoring
5–6	Recruiter CV results	Scored CVs visible in the dashboard
6–7	Interview infrastructure	Socket.IO gateway, sessions, transcripts
7–9	Technical interview AI	Realtime voice, adaptive agent, tldraw, Monaco
9–10	Integrity monitoring	Advisory event timeline
9–10	Soft skills interview	Behavioural agent reusing the engine
10–11	Final AI review	Combined report
10–12	Testing & hardening	Unit, integration, E2E, bias and load checks
11–12	Deployment & demo	Deployed MVP with demo data